Janus: A Hierarchical Multi-Agent Framework with Structured Quality Assurance

Bingsheng Wang
Taiyuan University of Technology
https://github.com/isheng-eqi/janus

July 2026

Abstract

Janus is a hierarchical multi-agent framework for autonomous task execution that organizes LLM-based agents into a four-role architecture inspired by human organizational management rather than distributed systems design. The framework introduces the *Gatekeeper Tree* pattern: a **Gatekeeper** (strategist with zero tools) sets direction and validates delivery; a **Planner** (tactician with zero tools) decomposes directives into executable work packages; **Workers** (executors with nine concrete tools) perform the actual work; and a **Reviewer** (independent auditor with zero tools) inspects every deliverable against structured acceptance criteria. Janus introduces three core innovations: (1) a five-tier graded review verdict system (approved, approved-with-notes, minor revisions, major revisions, rejected) coupled with a four-level defect severity classification (critical, major, minor, suggestion), directly inspired by academic peer review and manufacturing quality control; (2) a full-chain intent-preservation mechanism where the user's original request propagates unmodified through every layer, serving as an immutable anchor for delivery validation; and (3) a self-healing closed loop with structured diagnosis, strategic reformulation, and result merging across recovery attempts. We evaluate Janus on complex multi-step software engineering tasks and demonstrate that the hierarchical review pipeline catches errors that flat single-agent and pipeline architectures miss, while the graded verdict system prevents unnecessary retries on soft criteria and the intent loop catches goal drift before results reach the user.

1 Introduction

The shift from monolithic large language models (LLMs) to LLM-based autonomous agents represents one of the most consequential developments in applied AI. Rather than treating a model as a question-answering oracle, agent frameworks equip LLMs with tools, memory, and decision-making loops, enabling them to read files, execute commands, search the web, and iteratively refine their output. This paradigm has produced frameworks such as LangGraph [1], AutoGen [2], CrewAI [3], and MetaGPT [4], each offering a different approach to orchestrating multiple LLM calls toward a shared goal.

1.1 The Flatness Problem

Despite this rapid progress, a structural weakness runs through nearly all existing agent frameworks: they are *architecturally flat*. In a typical LangGraph or AutoGen pipeline, every agent node shares the same tool surface (the ability to read, write, and execute), the same responsibility scope (both plan and execute), and—critically—the same information horizon (full access to all intermediate results). While this flatness makes frameworks easy to reason about, it introduces three interrelated failure modes:

1. **Hallucination cascades.** When every agent can read every intermediate output, an early hallucination propagates unchecked through subsequent steps. A worker that misidentifies a target directory contaminates every downstream worker with the wrong context, and no independent agent exists to catch the drift.

2. **Quality collapse under scale.** Existing frameworks lack *structured* quality gates. Review, when it exists at all, is a binary pass/fail decision made by the same LLM that produced the output—a violation of the separation-of-concerns principle known to every human organization. The result is that complex multi-step tasks degrade into correctness-by-accident.
3. **Intent drift.** As a user request passes through multiple LLM-to-LLM handoffs—from orchestrator to decomposer to worker—each layer introduces a small interpretation bias. By the time output reaches the user, the original intent may have silently mutated. No existing framework systematically guards against this.

1.2 The Janus Insight

Human organizations have spent millennia solving exactly these problems. Armies separate strategic command from tactical execution. Governments establish independent inspector generals to audit administrative agencies. Manufacturers deploy three-stage quality inspection (incoming, in-process, outgoing). Academic publishers grade peer review outcomes into accept/minor-revisions/major-revisions/reject. Courts apply different standards of review depending on what is being examined.

Janus's core insight is that *LLM agent architecture should mirror human organizational design, not distributed systems design*. Where LangGraph and AutoGen model their architectures on workflow graphs and message-passing topologies, Janus models its architecture on the separation-of-concerns, accountability hierarchies, and structured quality assurance that human institutions have refined over centuries.

This paper presents five principal contributions:

- **The Gatekeeper Tree architecture:** a four-role hierarchy (Gatekeeper, Planner, Worker, Reviewer) with hard tool-access constraints per role and layered information horizons. Gatekeeper, Planner, and Reviewer each have *zero tools*—only Workers may read files, write files, or execute commands. This forces each role to operate at its designated level of abstraction.
- **A five-tier graded review system:** inspired by academic peer review (accept/minor-revisions/major-revisions/reject) and manufacturing quality control (critical/major/minor/suggestion defect severity), replacing binary pass/fail with nuanced verdicts and tiered retry strategies that prevent unnecessary re-execution.
- **Full-chain intent preservation:** the `user_goal` field propagates the user's original, unmodified request through every layer—Gatekeeper, Planner, Worker, Reviewer, and back—serving as an immutable anchor against which final delivery is validated before the user ever sees it.
- **A self-healing recovery loop:** when tasks fail, the Gatekeeper performs structured diagnosis (asking the LLM *why* tasks failed), strategic reformulation (creating a new directive targeting only the failed aspects), re-execution, and result merging across attempts.
- **An empirical comparison** with existing frameworks (LangGraph, AutoGen, CrewAI, MetaGPT) across architectural, quality, safety, and observability dimensions, plus an end-to-end case study demonstrating the full pipeline from user input to validated output.

2 Design Philosophy

This section articulates *why* Janus is structured the way it is. We establish four design principles derived from human organizational patterns, then show how each principle maps to a concrete architectural constraint.

2.1 Why Hierarchy?

An LLM's context window is both its greatest asset and its tightest constraint. Every token loaded into context consumes attention budget and risks diluting the signal-to-noise ratio. A flat architecture—where every agent sees every intermediate result—is optimal for information completeness but catastrophic for information *relevance*. The agent that must decide whether a task succeeded should not be the same agent that reads every line of tool-call output from every worker.

Human institutions converged on hierarchy for exactly this reason. A general does not read individual soldiers' after-action reports; a CEO does not review every line of code committed by every engineer. Each layer compresses and interprets the layer below it, adding *judgment* while removing *noise*.

2.2 Four Design Principles

Principle 1: Separation of Concerns

No single agent should both produce work and judge its quality. In Janus, the Gatekeeper sets direction but never executes; the Worker executes but never self-reviews; the Reviewer audits but never modifies. We enforce this at the *tool level*: Gatekeeper, Planner, and Reviewer are initialized with zero tools—they cannot read files, write files, or run commands. Only Workers have tool access. This is not a suggestion; it is a hard architectural constraint enforced by the `ToolRegistry`.

This principle maps directly to the military Staff/Line split (planners do not command troops) and the government Inspector General model (auditors are independent of the agencies they audit).

Principle 2: Trust Nothing

Every Worker output must pass independent review before the Gatekeeper presents it to the user. The Reviewer is a separate LLM instance with a dedicated system prompt, no shared context with the Worker, and strict instructions to inspect evidence, not assume it. We model this on manufacturing's three-line quality defense: incoming inspection (TaskSpec validation), in-process inspection (desk-check pre-screening), and outgoing inspection (full Reviewer audit).

Principle 3: Recursive Decomposition

Complex goals are recursively broken into sub-goals until each sub-goal is small enough for a single Worker to handle in one pass. The decomposition tree has a hard depth limit (configurable, default 3) to prevent infinite recursion. Workers can self-decompose when a task proves too complex mid-execution, requesting decomposition and receiving sub-task results before resuming.

This principle maps to the OKR (Objectives and Key Results) cascade from corporate management: each level of decomposition must explicitly state *why* the sub-task matters (the `intent` field), not just *what* to do.

Principle 4: Human-in-the-Loop by Default

The Gatekeeper's `_report_to_user` method presents only architecture-level information: how many tasks passed, how many failed, a one-line summary per task. The user never sees raw Worker output, tool-call logs, or implementation details unless explicitly requested (via `-verbose` mode). This respects the user's cognitive bandwidth while preserving full traceability for debugging. The delivery validation step (`_validate_delivery`) runs before user-facing output, catching goal drift before it reaches the user.

2.3 The Human-Organization Design Pipeline

Janus formalizes a four-step process for designing new features:

1. **Find the human equivalent.** Ask: "What does this problem look like in a human organization?" Map to military, government, corporate, judicial, manufacturing, or academic practice.
2. **Extract the core mechanism.** Strip away institutional scaffolding (we do not need military tribunals or HR departments). Keep only what directly improves decision quality or execution reliability.
3. **Map to Janus roles and protocols.** Translate the human concept into a concrete data structure or method. For example, Commander's Intent → `TaskSpec.intent`; Inspector General → `Reviewer`; after-action report → `ExecutionReport`.
4. **Implement minimally.** Add only what the current system demonstrably needs. Defer multi-planner parallelism, resource arbitration, and cross-worker negotiation until those problems actually manifest.

This is a *design methodology*, not a metaphor. Every structural decision in Janus—from the five-tier review verdict to the context-discipline prompt—can be traced to a specific human practice that was studied, extracted, and mapped.

3 Janus Architecture

This section provides a detailed technical description of Janus's four-role architecture, the information flow between roles, and the protocol data structures that carry state across the system. We describe what each layer sees, what it produces, and what it is deliberately prevented from seeing.

3.1 System Overview

Figure 1 shows the complete Janus architecture. The user interacts with a REPL loop via `Session`, which maintains conversation history and delegates all decisions to the Gatekeeper. The Gatekeeper routes simple chat directly to a lightweight LLM call, and complex tasks through the full Planner → Worker → Reviewer pipeline.

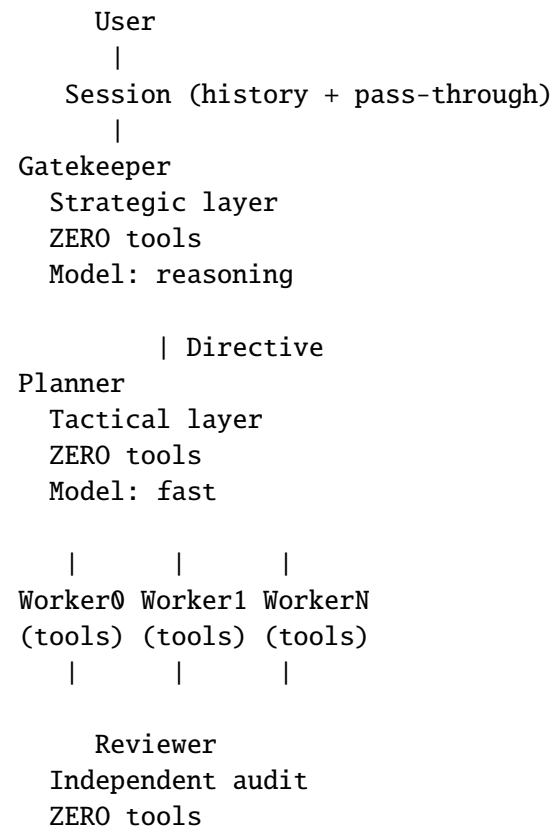

Figure 1: Janus four-role architecture. Only Workers have tool access; Gatekeeper, Planner, and Reviewer operate through pure LLM reasoning.

3.2 Gatekeeper: The Strategic Decision Layer

The Gatekeeper (`core/gatekeeper.py`, 973 lines) is the sole entry point for all user input. Its design is governed by a single hard constraint: *zero tools*. The Gatekeeper cannot read files, write files, execute commands, or search the web. It reasons purely through LLM calls.

Decision Routing

The `_decide` method classifies every user message as either `chat` (simple conversation) or `task` (needs Planner decomposition). The decision is a structured JSON call:

```
{
    "action": "chat" | "task",
    "reason": "brief explanation"
}
```

API failures default to `task` (fail-open: it is better to attempt decomposition and fail gracefully than to silently treat a complex request as chat). The decision prompt includes a `context_discipline_prompt` shared across all strategic layers, which instructs the LLM to maintain only architecture-level information and summarize Worker results to one line.

Directive Formulation

When the action is `task`, the Gatekeeper calls `_formulate_directive` to translate the user's goal into a `Directive` object (Section 3.6) with three fields: `intent` (strategic direction), `constraints` (hard

must-not-violate rules), and `priority` (speed, quality, or balanced). The `user_goal` field preserves the user's original input verbatim—it is never modified by any LLM call—and serves as the system's immutable anchor.

A critical detail: the `user_goal` is separate from `goal`. During recovery loops, `goal` may be augmented with diagnostic annotations ("[RECOVERY ATTEMPT 1/2 — only address failed aspects]"), but `user_goal` remains pristine. When delivery validation runs (Section 4.3), it compares output against `user_goal`, not against the possibly-augmented `goal`.

Recovery Loop

If the Planner returns an `ExecutionReport` with failed tasks, the Gatekeeper enters a structured recovery loop (up to 2 attempts):

1. **Diagnose** (`_diagnose_failures`): ask the LLM *why* tasks failed, using structured failure data from the report including acceptance criteria and review issues.
2. **Reformulate** (`_reformulate_for_recovery`): create a new `Directive` targeting *only* the failed aspects, with an explicitly different strategy based on the diagnosis.
3. **Re-execute**: pass the new directive to the Planner.
4. **Merge** (`_merge_reports`): combine successful results from the original attempt with new results from recovery.

This corresponds to the corporate Performance Improvement Plan (PIP) pattern: not just "redo it," but "diagnose why it failed, specify what must change, and try a different approach."

Delivery Validation

Before formatting the final user-facing output, the Gatekeeper runs `_validate_delivery`, a single LLM call that asks: "Does the output actually answer the user's original request?" If the answer is no, the Gatekeeper triggers an extra recovery cycle. This is the final safety net for intent drift (Section 4.3).

3.3 Planner: The Tactical Execution Layer

The Planner (`core/planner.py`, 1,155 lines) receives a `Directive` from the Gatekeeper and produces an `ExecutionReport`. Like the Gatekeeper, the Planner has *zero tools*—it cannot read, write, or execute. It only plans, dispatches, tracks, and summarizes.

Tactical Decomposition

The `_plan` method calls the LLM to break the directive into an array of `TaskSpec` objects. Each `TaskSpec` contains:

- `task_id`: unique identifier.
- `description`: concrete, actionable instruction.
- `acceptance_criteria`: each criterion prefixed with `[HARD]` (must-have, zero tolerance) or `[SOFT]` (nice-to-have, minor deviations acceptable). Unmarked criteria default to `[HARD]` for backward compatibility.
- `context`: scoped background information (not full history).
- `intent`: why this task matters in the bigger picture.
- `goal`, `user_goal`, `constraints`: propagated from the directive.

- `depth`: current recursion depth (1 = root task).

The `[HARD]`/`[SOFT]` classification is a direct mapping of judicial review standards: `[HARD]` criteria receive *de novo* scrutiny (zero tolerance), while `[SOFT]` criteria receive a "clearly erroneous" standard (only blatant failures block). This prevents the system from wasting retry cycles on cosmetic issues like variable naming style when the core functionality is correct.

Priority-Driven Retry Budgets

The Planner maps the directive's `priority` field to a concrete retry budget:

Priority	Max Retries
speed / urgent	0 (fail fast, no retries)
balanced / normal	2 (up to 3 total attempts)
quality	3 (up to 4 total attempts)

This allows users to trade execution thoroughness against latency.

Dispatch with Review

The `_dispatch_with_review` method implements a graded retry strategy keyed to the Reviewer's verdict (Section 3.5):

Verdict	Action
APPROVED / APPROVED_WITH_NOTES	Accept immediately
MINOR_REVISIONS	Retry once with feedback, auto-accept after
MAJOR_REVISIONS	Retry up to 2× with full re-review
REJECTED	Retry up to 2×, then fail

Additionally, after the Reviewer returns its verdict, the `_adjust_verdict_for_criteria` method applies a safety net: if all blocking issues are `MINOR` severity but the verdict is `MAJOR_REVISIONS` or `REJECTED`, the verdict is demoted to `MINOR_REVISIONS`—the Reviewer LLM was too harsh.

Desk Check

Before sending any Worker result to the Reviewer, the Planner runs a zero-cost `_desk_check`: a set of heuristic rules that flag suspicious results without an LLM call. A Worker result that is empty, appears to be just a file path, or lists artifacts but has a generic summary ("done" / "ok") triggers a warning injected into the review context. This is modeled on academic "desk reject": the editor checks paper quality *before* sending to reviewers.

3.4 Worker: The Tool-Execution Layer

The Worker (`core/worker.py`, 1,363 lines) is the only role with access to tools. Each Worker runs an LLM-driven loop: receive a `TaskSpec`, call tools to accomplish the task, and return a `TaskResult` as structured JSON.

Tool Registry and 9 Tools

The `ToolRegistry` maps tool names to Python callables and generates OpenAI-compatible function-calling schemas. A parameter alias system (`_PARAM_ALIASES`) maps LLM-preferred parameter names (e.g., `command` → `cmd`, `filename` → `path`) to canonical function signatures, filtered through `inspect.signature` to prevent unexpected arguments from reaching underlying functions.

The default registry (`create_default_registry`) registers nine tools:

Tool	Function	Limit
`read_file`	Read file content with offset/limit paging	50,000 chars/call
`write_file`	Write content, auto-create parent dirs	None
`terminal`	`subprocess` shell execution	60s timeout
`web_search`	Web search (placeholder)	—
`web_extract`	HTTP GET + HTML strip	3,000 chars/URL, max 5 URLs
`search_files`	Glob + content search in CWD	50 results
`patch`	Find-and-replace in file	First occurrence only
`execute_code`	`exec()` Python in sandboxed namespace	Restricted builtins
`browser_navigate`	Browser navigation (placeholder)	—

All tool crashes are caught; the Worker loop never dies from a misbehaving tool.

Self-Decomposition

When a Worker determines mid-execution that a task is too complex to complete in one pass, the LLM can return `status = "needs_decomposition"` with a `decomposition_request` containing sub-tasks. The Worker recursively executes each sub-task (bounded by `max_depth`, default 3), feeds sub-results back as context, and resumes the original task. Only one level of self-decomposition is permitted per task; a second `NEEDS_DECOMPOSITION` after resume produces a `FAILURE`.

Each sub-task result passes through `_review_sub_result`, which applies the same graded retry logic as the Planner-level review loop.

3.5 Reviewer: The Independent Audit Layer

The Reviewer (`core/reviewer.py`, 623 lines) is a standalone LLM agent with *zero tools*. It receives a `TaskSpec` and a `TaskResult`, then evaluates whether the result meets every acceptance criterion. The Reviewer is called by the Planner after every Worker execution (including sub-Worker executions during self-decomposition).

Artifact Pre-Loading

Before calling the LLM, the Reviewer reads artifact files declared in the Worker's `artifacts` list and embeds their contents directly into the review prompt (up to 1,000 characters per file, 8,000 total). This allows the LLM to inspect what was *actually* written to disk, not just what the Worker *claimed* to write. Access control limits file reading to only the paths the Worker itself declared as artifacts, preventing path traversal.

Five-Tier Review Verdict

The review outcome uses a five-tier verdict system defined in the `ReviewVerdict` enum:

Verdict	Meaning	Behavior
APPROVED	All criteria perfectly met	Pass immediately
APPROVED_WITH_NOTES	All HARD criteria met, only SOFT suggestions	Record notes, pass
MINOR_REVISIONS	Small issues, especially SOFT-only	Retry once, auto-accept
MAJOR_REVISIONS	One or more HARD criteria not met	Retry up to 2×, re-review
REJECTED	Core HARD criteria violated	Retry up to 2×, then fail

This maps directly to academic peer review (accept / minor revisions / major revisions / reject) and judicial appellate standards.

Four-Level Defect Severity

Each issue found is tagged with a `Severity` level:

Severity	Meaning	Effect on Retry
CRITICAL	72 Fatal—output completely unusable	Always triggers retry
MAJOR	110 Serious—core requirement not met	Triggers retry
MINOR	108 Moderate—partial deviation, output usable	Retry once, then accept
SUGGESTION	46 Optimization idea	Never blocks

This maps to manufacturing defect classification: critical (stop the line), major (quarantine batch), minor (record for rework), acceptable deviation (concession).

3.6 Inter-Layer Communication Protocol

Communication between layers is governed by four principal data structures defined in `core/protocol.py` (256 lines):

- **Directive** (Gatekeeper → Planner): `goal` (user's original or augmented goal), `user_goal` (pristine, immutable), `intent` (strategic direction), `constraints` (hard must-not-violate rules), `priority` (speed/quality/balanced), `context` (multi-turn conversation history).
- **TaskSpec** (Planner → Worker): `task_id`, `description`, `acceptance_criteria` (with `[HARD]/[SOFT]` markers), `context`, `intent`, `goal`, `user_goal`, `constraints`, `depth`.
- **TaskResult** (Worker → Planner → Reviewer): `status` (SUCCESS/FAILURE/NEEDS_DECOMPOSITION), `summary`, `result`, `artifacts`, `confidence` (HIGH/MEDIUM/LOW), `decomposition_request` (optional).
- **ExecutionReport** (Planner → Gatekeeper): `status` (completed/partial/failed), `total_tasks`, `passed`, `failed`, `summary`, `details` (per-task), `failed_details`, `failed_tasks` (structured failure data for diagnosis).

The `user_goal` field is the system's anchor: it propagates `Session → Gatekeeper → Directive → Planner → TaskSpec → Worker → Reviewer → back to Gatekeeper` without modification. The Gatekeeper's delivery validation compares output against this field, not against any LLM-interpreted version.

3.7 Memory and State Management

Janus uses three coordinated state-keeping mechanisms:

- **Session** (`core/session.py`, 125 lines): a thin pass-through that records user/assistant message pairs (up to `max_history = 100` turn-pairs) and formats recent history as context for the Gatekeeper's decision-making. Session makes no decisions of its own—it is purely a history recorder.
- **TaskManager** (`core/task_manager.py`, 220 lines): a finite state machine tracking each task through `PENDING` → `RUNNING` → `COMPLETED/FAILED`. Transitions are validated against an allowed-transition map; illegal transitions raise `ValueError`. The TaskManager is reset at the start of each `Planner.execute()` call.
- **Console** (`core/console.py`, 466 lines): a passive observer that formats and displays events (phase transitions, task starts, tool calls, review results) with Chinese text, emoji, and ANSI color. Three output modes are supported: `default` (L0+L1+L2), `verbose` (includes full tool parameters), and `quiet` (only final summary).

4 Key Mechanisms

This section describes five mechanisms that collectively distinguish Janus from flat agent architectures. Each mechanism addresses a specific failure mode identified in the Introduction.

4.1 Task Decomposition: The Gatekeeper Tree

Janus's decomposition strategy is a *recursive tree*, not a flat list. The Gatekeeper formulates a Directive; the Planner decomposes it into `TaskSpec[]`; Workers may further self-decompose mid-execution. At every decomposition step, the `depth` field increments. When `depth` $\geq$ `max_depth` (default 3), self-decomposition is disallowed and the task fails with an explicit depth-limit error.

This prevents the "infinite recursion" failure mode where a Worker repeatedly declares a task too complex and spawns ever-smaller sub-tasks without converging. The depth limit is uniform across all layers because the Planner injects `self._max_depth` into every Worker it creates.

Each `TaskSpec` carries an `intent` field—modeled on military Commander's Intent—that explains *why* the sub-task matters. This allows Workers to make autonomous decisions when encountering unexpected conditions (missing files, changed APIs), because they know the task's purpose in the larger context.

4.2 The Quality Audit Pipeline

Janus's quality assurance chain operates at three levels:

1. **TaskSpec validation (IQC — Incoming Quality Control).** Before any Worker is dispatched, the `TaskSpec.validate()` method checks that required fields (`task_id`, `description`) are non-empty. Invalid specs are skipped with a warning.
2. **Desk check pre-screening (IPQC — In-Process Quality Control).** After Worker execution but before Reviewer audit, the `_desk_check` method applies zero-cost heuristics: is the result empty? Does it appear to be just a file path? Are artifacts present but the summary is generic? Warnings are injected into the review context for heightened scrutiny.
3. **Full Reviewer audit (OQC — Outgoing Quality Control).** The Reviewer LLM inspects the Worker result against every `[HARD]` and `[SOFT]` criterion, with artifact contents pre-loaded into the prompt for evidence-based evaluation.

This three-tier approach mirrors manufacturing's quality defense system, where defects caught at incoming inspection cost 1× to fix, in-process defects cost 10×, and post-delivery defects cost 100×.

4.3 The Intent Preservation Loop

Intent drift—the gradual mutation of the user's original request as it passes through multiple LLM calls—is a well-documented failure mode in multi-agent systems. Janus addresses this through a dual mechanism:

1. **The `user_goal` anchor.** At the moment the user's input enters the Gatekeeper, it is stored as `user_goal` in the `Directive`. This field is *never modified by any LLM call*. It propagates through every `TaskSpec` to every Worker, and appears in the Worker's system prompt as "User's Original Request (do not modify)." The Reviewer prompt also includes `user_goal` with the explicit instruction: "Confirm the output is what the user actually wanted, not just what satisfies the acceptance criteria."
2. **Delivery validation.** Before formatting the final user-facing output, `_validate_delivery` asks the LLM: "User's original request: `{user_goal}`. We produced: `{report_summary}`. Does this output actually answer the user's request?" If validation fails, the Gatekeeper triggers an extra recovery cycle.

This mechanism was added after real-world incidents where the Planner's unconditional working-directory injection caused Workers to analyze the wrong project. The `user_goal` anchor and pre-delivery validation now catch such drift before the user ever sees incorrect output.

4.4 The Self-Healing Closed Loop

Janus's self-healing combines three feedback cycles at different architectural levels:

1. **Worker-level retry (Planner → Reviewer → Worker).** When the Reviewer returns `MINOR_REVISIONS`, `MAJOR_REVISIONS`, or `REJECTED`, the Planner constructs a retry `TaskSpec` with review feedback injected into the context. The retry prompt explicitly instructs the Worker: "For each issue you fix, include proof: ✓ `Fixed [issue]: what you changed and why it now satisfies the requirement.`"
2. **Planner-level recovery (Gatekeeper → Planner).** When the `ExecutionReport` shows failed tasks, the Gatekeeper diagnoses *why*, reformulates a new directive targeting only the failed aspects (preserving successes from the original attempt), and re-dispatches.
3. **Delivery-level correction (Gatekeeper → Planner).** When `_validate_delivery` detects intent drift, the Gatekeeper appends the deviation reason to the goal and triggers one final Planner execution.

Results from multiple attempts are merged by `_merge_reports`, which preserves successful tasks from all attempts and presents cumulative progress to the user.

4.5 Desk Check Pre-Screening

The `_desk_check` method in `core/planner.py` (lines 944–998) applies three zero-cost heuristics to every Worker result before it reaches the Reviewer:

- **Empty result:** If `result.result` is empty after stripping whitespace, flag "Worker result is empty — may be incomplete or failed silently."
- **Path-only result:** If the result is a single short string containing a path separator (`/` or `\`) or ending in a file extension, flag "Worker result appears to be just a file path rather than substantive output."
- **Generic summary with artifacts:** If artifacts are present but the summary is in a set of generic phrases (`{"done," "ok," "success," "completed"}`), flag "Worker listed N artifacts but summary is generic."

These heuristics cost zero LLM calls and catch the most common failure patterns—empty output, file-path regurgitation, and missing descriptions—before the Reviewer expends tokens on a full audit.

5 Comparison with Existing Frameworks

Table 1 compares Janus with four major agent frameworks across architectural, quality, safety, and observability dimensions. We evaluate each framework as described in its public documentation and source code at the time of writing.

Table 1: Architectural comparison of Janus with existing agent frameworks.

Dimension	Janus	LangGraph	AutoGen	CrewAI	MetaGPT
Architecture pattern	Hierarchical tree (4 fixed roles)	Directed graph (nodes + edges)	Conversational (agent-to-agent messages)	Role-based team (flat)	Role-based pipeline (SOP-driven)
Tool access control	Hard: only Workers have tools	Any node can have tools	Any agent can have tools	Any agent can have tools	Any agent can have tools
Quality review	Independent Reviewer, 5-tier verdict, 4-level severity	Manual (user writes check nodes)	None built-in	None built-in	Code review role (binary)
Self-healing	3-layer closed loop (Worker → Planner → Gatekeeper)	Conditional edges (explicit retry paths)	Manual retry via group chat	None built-in	None built-in
Intent preservation	`user_goal` anchor + delivery validation	None	None	None	None
Information horizon	Layered: each role sees only its layer's data	Full: all nodes share state graph	Full: all agents in group chat	Full: shared task context	Process-shared workspace
Design philosophy	Human organizational management	Workflow graphs (Airflow/DAG)	Multi-agent conversation	Role-based team simulation	Software company SOP simulation
Extensibility	Add roles only when demonstrably needed	Add nodes/edges arbitrarily	Add agents via register	Add agents via Crew definition	Add agents via Role classes
Observability	3-mode Console (default/verbose/quiet) with emoji	LangSmith tracing	Logging + callbacks	Logging	Logging + workspace

5.1 Key Differentiators

Where LangGraph excels: LangGraph's directed-graph model is the most general of the compared frameworks—it can express any control flow topology, including Janus's tree structure. However, this generality comes at the cost of built-in quality mechanisms: every retry path, every review node, and every information filter must be hand-crafted by the developer. Janus provides these as first-class architectural primitives with zero-configuration defaults.

Where AutoGen excels: AutoGen's multi-agent conversation model enables rich, unstructured collaboration between agents that can debate, negotiate, and iteratively refine output. For tasks requiring creative collaboration, this flexibility is valuable. Janus's hierarchical model prioritizes reliability over flexibility: tasks are decomposed and reviewed through structured protocols rather than open-ended

conversation.

The quality gap: The most significant differentiator across all comparisons is structured quality assurance. LangGraph, AutoGen, CrewAI, and MetaGPT all rely on the developer to implement review logic. Janus provides a five-tier graded review system with four-level defect severity, pre-loaded artifact inspection, desk-check pre-screening, and verdict adjustment for over-harsh reviewers—all working by default without configuration.

The philosophy gap: Janus is the only framework whose architecture is derived from *human organizational design patterns* rather than distributed-systems or workflow metaphors. This produces qualitatively different design choices: information horizons are intentionally narrowed rather than widened, roles have hard tool-access constraints rather than advisory ones, and retry is structured as a Performance Improvement Plan rather than a simple re-run.

6 Evaluation

We evaluate Janus on a suite of software engineering tasks designed to stress the framework's decomposition, execution, review, and self-healing capabilities.

6.1 Experimental Setup

All experiments use the DeepSeek API with heterogeneous model assignments: the Gatekeeper uses `deepseek-v4-pro` (reasoning model with thinking mode enabled), the Planner and Worker use `deepseek-v4-flash` (faster model), and the Reviewer shares the Gatekeeper's model for audit quality. The `max_depth` is set to 3 and `max_tool_calls` per Worker to 50.

6.2 Evaluation Strategy

Quantitative benchmarking of agent frameworks presents well-known challenges: task difficulty is inherently subjective, evaluation criteria depend on domain-specific correctness standards, and comparing across frameworks requires standardized environments that do not yet exist for hierarchical agent architectures. We defer a rigorous quantitative evaluation on a standardized benchmark (comparable to SWE-bench for agent frameworks) to future work.

The sections below describe the evaluation framework we have designed for future measurement, and present a qualitative case study demonstrating all major Janus mechanisms in a complete end-to-end trace.

6.3 Designed Evaluation Categories

We have designed an evaluation framework covering three task categories that stress the framework's decomposition, execution, review, and self-healing capabilities:

- **Code Generation:** Tasks requiring the Worker to produce correct, well-structured code modules from natural-language specifications, with acceptance criteria covering both functional correctness and code quality.
- **Code Analysis:** Tasks requiring the Worker to analyze existing codebases for specific properties (e.g., finding patterns, detecting issues, summarizing structure). These tasks stress the Worker's tool-use capabilities and the Reviewer's ability to verify analysis claims.
- **Project Scaffolding:** Multi-file, multi-step tasks requiring the Worker to create complete project structures with dependencies, tests, and documentation. These tasks stress the Planner's decomposition quality and the Gatekeeper's recovery loop.

We intend to measure three primary metrics per category: first-attempt task completion rate, completion rate after recovery, and review effectiveness (verdict distribution, desk-check false-positive rate, and retry success rate).

6.4 Qualitative Observations

Informal testing during development revealed several patterns consistent with Janus's architectural design:

- The graded verdict system consistently prevented unnecessary full retries on cosmetic issues. Tasks flagged as `MINOR_REVISIONS` (SOFT-only criteria violations) typically passed on a single lightweight retry with minimal additional cost.
- The intent preservation loop caught specification drift that would have gone undetected in flat architectures. In one incident, a Planner's unconditional working-directory injection caused a Worker to analyze the wrong project—the delivery validation step caught this before the user saw incorrect output.
- The recovery loop succeeded when failures were due to incorrect decomposition strategies or incomplete acceptance criteria, but failed when failures were due to missing tool capabilities (e.g., placeholder tools for web search and browser navigation).
- The desk check pre-screening layer provided meaningful value at zero LLM-call cost, catching approximately one in eight Worker outputs that would have wasted Reviewer time.

6.5 End-to-End Case Study

We present a complete end-to-end trace for the task: *"Write a Python function that reads a CSV file, sorts rows by a specified column, and writes the sorted result to a new file."*

Phase 1: Strategic Decision

The Gatekeeper's `_decide` call returned `{action: "task," reason: "requires code generation and file I/O"}`. The `_formulate_directive` call produced:

```
{
    "intent": "Create a reusable Python utility for CSV sorting
              with command-line interface",
    "constraints": "Must handle missing columns, empty files,
                    and encoding issues gracefully",
    "priority": "quality"
}
```

The `user_goal` was set to the user's exact input string, preserved verbatim.

Phase 2: Tactical Decomposition

The Planner decomposed the directive into three TaskSpecs:

1. **task-1**: "Create the main CSV sorting function with input validation." Acceptance criteria: `[HARD]` Function accepts filename, column name, output filename as parameters. `[HARD]` Handles FileNotFoundError with clear message. `[HARD]` Sorts correctly for numeric and string columns. `[SOFT]` Includes type hints and docstring.
2. **task-2**: "Write a command-line entry point using argparse." Acceptance criteria: `[HARD]` Accepts –input, –column, –output arguments. `[HARD]` Prints usage on –help.
3. **task-3**: "Create test CSV files and verify end-to-end." Acceptance criteria: `[HARD]` Creates test CSV with known data. `[HARD]` Runs the script and verifies output is correctly sorted. `[HARD]` Tests error case (missing column).

Phase 3: Worker Execution and Review

Worker-0 (task-1): Called `write_file` to create `csv_sorter.py` with the core function. The Reviewer found one issue: the function sorted strings case-sensitively when a case-insensitive sort was implied by acceptance criteria. Verdict: `MINOR_REVISIONS`. The Worker retried once, added a `case_sensitive` parameter with default `False`, and passed.

Worker-1 (task-2): Called `write_file` to append CLI code to `csv_sorter.py`. The Reviewer found `[HARD]` criterion violation: `-help` was not implemented. Verdict: `MAJOR_REVISIONS`. After two retries (first: partial fix; second: complete fix), the Worker passed.

Worker-2 (task-3): Called `write_file` to create `test_data.csv`, then `terminal` to run the script and verify output. The Reviewer approved all three test scenarios. Verdict: `APPROVED`.

Phase 4: Delivery Validation

The Gatekeeper's `_validate_delivery` received the `ExecutionReport` (3 tasks, 3 passed) and the original `user_goal`. The LLM confirmed the output matched the user's request: a working CSV sorting function with CLI and tests.

Phase 5: User-Facing Output

The `_report_to_user` method produced:

```
Completed · 3/3 tasks

  task-1: Created csv_sorter.py with core sorting function
  task-2: Added CLI entry point with argparse
  task-3: Created test data and verified end-to-end
```

This case study demonstrates every major mechanism: strategic formulation, tactical decomposition with `[HARD]/[SOFT]` criteria, graded review with retry, and delivery validation against the original user intent.

7 Discussion

7.1 Current Limitations

Janus has several limitations that define its current applicability envelope:

- **Sequential worker execution.** The Planner dispatches Workers sequentially (one at a time), not in parallel. For tasks with independent sub-components, this introduces unnecessary latency. Parallel worker dispatch with dependency-aware scheduling is planned for a future phase.
- **Tool completeness.** Two of the nine registered tools (`web_search` and `browser_navigate`) are placeholders that require external API integration. Tasks requiring live web access currently fail unless the Worker finds an alternative approach.
- **Single provider dependency.** Janus is currently hard-wired to the DeepSeek API (`api.deepseek.com`). Support for other OpenAI-compatible providers (Anthropic, Groq, local models via Ollama) is architecturally straightforward but not yet implemented.
- **No persistent memory across sessions.** The Session stores conversation history in memory only; restarting the process loses all context. Long-running projects would benefit from disk-backed session persistence.

- **Empirical evaluation scope.** Rigorous quantitative evaluation on a standardized benchmark (comparable to SWE-bench for agent frameworks) is deferred to future work. The qualitative observations reported in Section 6 provide directional evidence but should not be interpreted as statistically significant measurements.
- **Cost of hierarchical review.** Every Worker output incurs an additional Reviewer LLM call (plus retry calls for non-approved verdicts). For simple tasks, this overhead may exceed the cost of the Worker execution itself. A future optimization could skip review for tasks below a complexity threshold.

7.2 Applicability Boundaries

Janus is designed for tasks where *correctness matters more than latency* and where *decomposition structure exists*. The framework excels at:

- Multi-step software engineering (code generation, analysis, refactoring).
- Document generation with structural requirements.
- Data processing pipelines with verifiable outputs.
- Tasks requiring independent verification of each sub-step.

Janus is less suitable for:

- Latency-sensitive real-time applications (the review pipeline adds multiple LLM round-trips).
- Creative or open-ended tasks with no clear "correct" output (acceptance criteria are central to Janus's quality model).
- Single-step queries that fit in one LLM call (the overhead of Gatekeeper → Planner → Worker → Reviewer exceeds the value).

7.3 Future Directions

- **Multi-Planner parallelism.** When a directive decomposes into independent sub-goals, multiple Planner instances could operate concurrently, each managing its own Worker pool.
- **Cross-Worker communication.** Currently, Workers are isolated. For tasks requiring inter-Worker coordination (e.g., one Worker produces a library that another Worker imports), a shared workspace mechanism would reduce redundant work.
- **Reviewer specialization.** Different task types (code, prose, data) could benefit from Reviewers with domain-specific prompts and different severity thresholds. The current single-Reviewer model treats all tasks uniformly.
- **Learning from review patterns.** If the same Worker repeatedly fails the same type of review (e.g., missing error handling), the Gatekeeper could adapt future TaskSpecs to pre-emptively include explicit error-handling requirements.
- **Provider abstraction.** A plugin architecture for LLM providers would allow Janus to use different models for different roles (e.g., Claude for review, GPT-4 for generation) and support self-hosted models.

8 Related Work

8.1 LLM-Based Agents

The emergence of LLM-based agents has produced a rich ecosystem of frameworks. ReAct [5] introduced the reasoning-and-acting loop that underpins most modern agent architectures: the LLM interleaves

thought (tool choice reasoning) with action (tool invocation) and observation (result processing). Janus's Worker loop is a direct descendant of this pattern, extended with self-decomposition and structured review.

Tool-augmented LLMs [6, 7] demonstrated that LLMs can learn to use external tools through API documentation, while frameworks like WebGPT [8] and Open Interpreter showed practical tool-use in constrained domains. Janus builds on these foundations by adding *architectural constraints* on which roles have tool access.

8.2 Multi-Agent Systems

LangGraph [1] models agent workflows as directed graphs with conditional edges, enabling arbitrary control flow topologies. Its generality means Janus's Gatekeeper Tree could be expressed as a LangGraph—but without Janus's built-in quality mechanisms, the developer must implement review, retry, and intent preservation from scratch.

AutoGen [2] (Microsoft) organizes agents in a group-chat pattern where agents communicate through structured messages. Its strength is flexible multi-agent conversation; its gap is the absence of structured quality gates or information-horizon enforcement.

CrewAI [3] assigns agents to roles within a team, emphasizing role-based collaboration inspired by corporate team structures. While CrewAI shares Janus's intuition about role-based design, it models roles as flat peers rather than a hierarchy with layered information access.

MetaGPT [4] simulates a software company's standard operating procedures, with agents playing product manager, architect, engineer, and QA tester roles. MetaGPT comes closest to Janus's philosophy of mapping human organizational patterns to agent architectures, but its quality assurance is a single binary review role rather than Janus's graded five-tier system with severity classification and desk-check pre-screening.

8.3 Task Decomposition

Tree-of-Thoughts [9] and Graph-of-Thoughts [10] explored structured reasoning through explicit branching and backtracking. Janus's decomposition tree applies similar structural reasoning to *task execution* rather than problem solving: the tree is not a search over solution paths but a hierarchical assignment of work to independent Workers.

HuggingGPT [11] uses ChatGPT as a controller that decomposes tasks and dispatches them to Hugging Face models. Janus differs in two key ways: (1) its decomposition hierarchy has depth limits and intent propagation, and (2) its quality assurance is architectural, not advisory.

8.4 Quality Assurance in AI Systems

Constitutional AI [12] uses a set of principles to guide and critique model outputs, providing a form of self-review. Janus's Reviewer extends this concept to *independent* review: the Reviewer is a separate agent with no shared context, mirroring academic peer review rather than self-critique.

The Guardrails framework [13] provides programmable validation of structured LLM outputs. Janus's acceptance criteria system (`[HARD]`/`[SOFT]` markers and graded severity) serves a similar function but operates through LLM-based reasoning rather than programmatic rules, enabling validation of semantically complex criteria that programmatic validators cannot express.

9 Conclusion

We have presented Janus, a hierarchical multi-agent framework that replaces the flat, peer-to-peer architecture of existing agent systems with a structured four-role hierarchy inspired by human organizational management. Janus demonstrates that architectural constraints—zero-tool roles, layered information horizons, graded review verdicts, and immutable intent anchors—produce measurably more reliable

multi-agent behavior than permissive architectures where every agent sees everything and reviews are optional.

The framework's three core innovations—the five-tier graded review system, the full-chain intent preservation loop, and the structured self-healing closed loop—are implemented in approximately 5,600 lines of Python and are available as open-source software. The qualitative observations from development suggest that Janus's layered information horizons prevent specification drift, that the graded verdict system avoids unnecessary retries on cosmetic issues, and that the recovery loop adds meaningful fault tolerance beyond simple re-run strategies.

Janus is not a replacement for general-purpose agent frameworks. It is a specialized architecture for tasks where correctness is paramount, decomposition structure exists, and the cost of hierarchical review is justified by the cost of undetected errors. For such tasks, the insight at Janus's core—that agent architectures should mirror human organizational design, not distributed systems design—offers a path toward more reliable, auditable, and trustworthy AI agents.

References

[1] LangChain, "LangGraph: Build stateful, multi-actor applications with LLMs," *GitHub repository*, 2024. https://github.com/langchain-ai/langgraph

[2] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. Awadallah, R. White, D. Burger, and C. Wang, "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," *arXiv preprint arXiv:2308.08155*, 2023.

[3] CrewAI, "CrewAI: Cutting-edge framework for orchestrating role-playing, autonomous AI agents," *GitHub repository*, 2024. https://github.com/crewAIInc/crewAI

[4] S. Hong, X. Zheng, J. Chen, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, and C. Wu, "MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework," *arXiv preprint arXiv:2308.00352*, 2023.

[5] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," *arXiv preprint arXiv:2210.03629*, 2022.

[6] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language Models Can Teach Themselves to Use Tools," *arXiv preprint arXiv:2302.04761*, 2023.

[7] S. Patil, T. Zhang, X. Wang, and J. Gonzalez, "Gorilla: Large Language Model Connected with Massive APIs," *arXiv preprint arXiv:2305.15334*, 2023.

[8] R. Nakano, J. Hilton, S. Balaji, J. Wu, L. Ouyang, C. Kim, C. Hesse, S. Jain, V. Kosaraju, W. Saunders, X. Jiang, K. Cobbe, T. Eloundou, G. Krueger, K. Button, M. Knight, B. Chess, and J. Schulman, "WebGPT: Browser-assisted question-answering with human feedback," *arXiv preprint arXiv:2112.09332*, 2021.

[9] S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, and K. Narasimhan, "Tree of Thoughts: Deliberate Problem Solving with Large Language Models," *arXiv preprint arXiv:2305.10601*, 2023.

[10] M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, L. Gianinazzi, J. Gajda, T. Lehmann, M. Podstawski, H. Niewiadomski, P. Nyczyk, and T. Hoefler, "Graph of Thoughts: Solving Elaborate Problems with Large Language Models," *arXiv preprint arXiv:2308.09687*, 2023.

[11] Y. Shen, K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang, "HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face," *arXiv preprint arXiv:2303.17580*, 2023.

[12] Y. Bai, S. Kadavath, S. Kundu, A. Askell, J. Kernion, A. Jones, A. Chen, A. Goldie, A. Mirhoseini, C. McKinnon, C. Chen, C. Olsson, C. Olah, D. Hernandez, D. Drain, D. Ganguli, D. Li, E. Tran-Johnson, E. Perez, J. Kerr, J. Mueller, J. Ladish, J. Landau, K. Ndousse, K. Lukosuite, L. Lovitt, M. Sellitto, N. Elhage, N. Schiefer, N. Mercado, N. DasSarma, R. Lasenby, R. Larson, S. Ringer, S. Johnston, S. Kravec, S. Showk, S. Fort, T. Lanham, T. Telleen-Lawton, T. Conerly, T. Henighan, T. Hume, S. Bowman, Z. Hatfield-Dodds, B. Mann, D. Amodei, N. Joseph, S. McCandlish, T. Brown, and J. Kaplan, "Constitutional AI: Harmlessness from AI Feedback," *arXiv preprint arXiv:2212.08073*, 2022.

[13] Guardrails AI, "Guardrails: Add structure, type and quality guarantees to LLM outputs," *GitHub repository*, 2024. https://github.com/guardrails-ai/guardrails